

E-Commerce Data Warehouse

Student: [Abdullah Shamsan]

Date: 29/ 4 / 2026

1. مقدمة وهدف المشروع.

يهدف هذا المشروع إلى تصميم وتنفيذ مستودع بيانات (Data Warehouse) قابل للإنتاج-Production (Ready) لمتجر إلكتروني، باستخدام مجموعة بيانات Olist البرازيلية. يركز الحل على تمكين تقارير تحليلية عالية الأداء تجيب عن أسئلة الأعمال الحرجة مثل: اتجاهات المبيعات، العملاء الأكثر قيمة، أداء التسليم، والمنتجات/الفئات الأكثر ربحية.

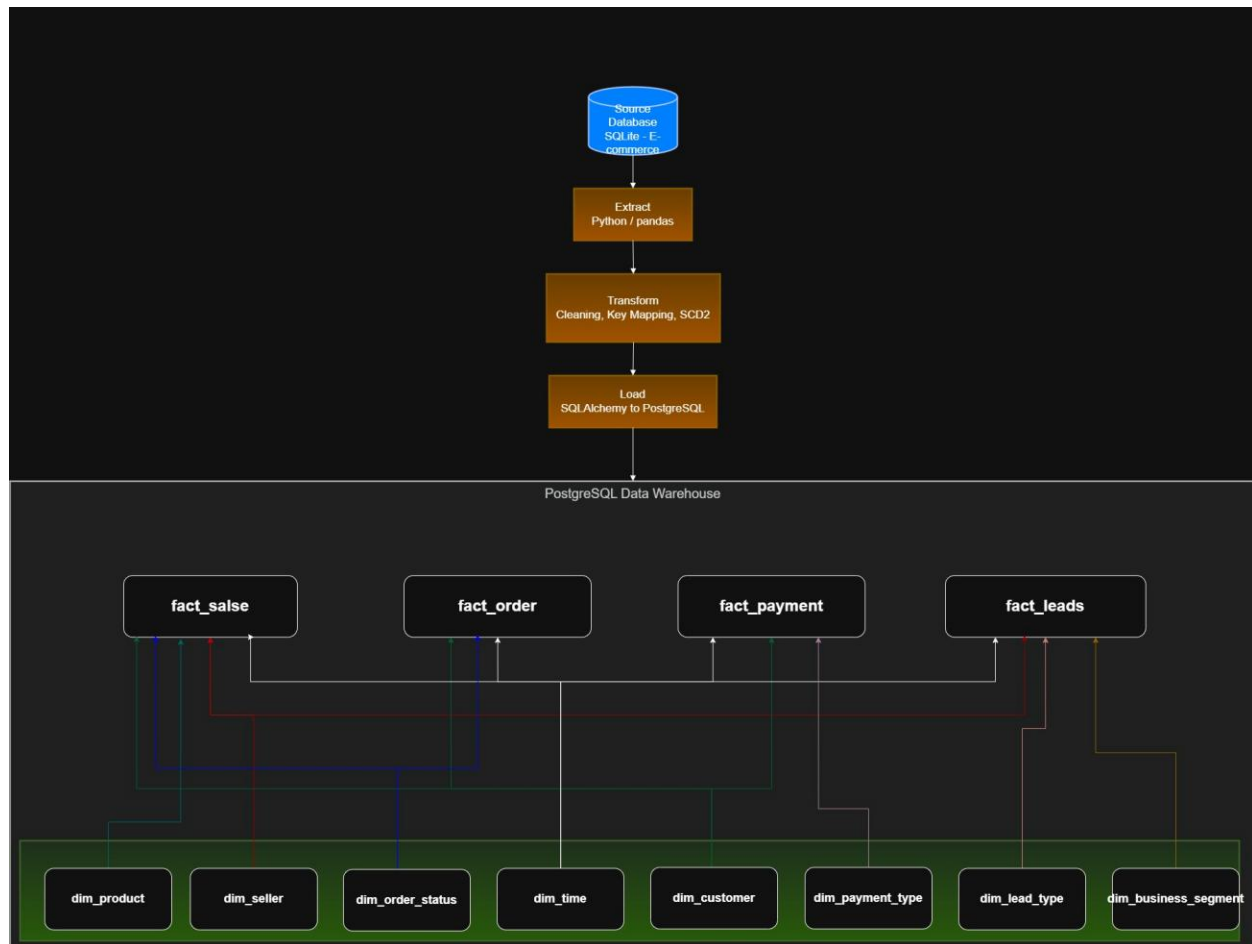
تم بناء المستودع وفق منهجية معتمدة في نمذجة البيانات، مع خط أنابيب ETL قوي، ومجموعة من الاستعلامات الجاهزة للاستخدام من قبل محلي الأعمال.

2. تصميم مستودع البيانات (Data Warehouse Design).

1. Galaxy Schema .

- **تعدد العمليات التجارية:** بيانات Olist لا تقتصر على عملية بيع واحدة، بل تشمل عمليات بيع، وتوصيل (شحن)، ودفع، وتقييم، وتسويق. كل عملية لها تفاصيل (Granularity) الخاصة. مخطط Galaxy يسمح بوجود عدة جداول حقائق (Fact Tables) تتشارك أبعاداً مطابقة (Conformed Dimensions) دون الدمج القسري الذي يسبب تكرار البيانات أو تشويه المقاييس.
- **أسئلة التحليل متعددة الأبعاد:** جميع الأسئلة المطلوبة (المبيعات عبر الزمن، العملاء، التسليم، المنتجات) تتطلب تجمعات (Aggregations) على أبعاد مثل الزمن، العميل، المنتج. المخطط النجمي أو المجري يقدم ببساطة في الفهم وسرعة في الأداء.
- **المرونة والتوسع:** يمكن إضافة حقائق جديدة (مثلاً: المخزون، العروض) أو أبعاد جديدة دون إعادة هيكلة المستودع، وهو ما يدعمه مخطط Galaxy بشكل ممتاز.
- **الفصل بين التفاصيل:** لا يمكن دمج المبيعات (على مستوى بند الطلب) مع التقييمات (على مستوى الطلب) في جدول واحد دون تكرار التقييم لكل بند، مما يحرف المتوسطات. لذا فصلناها في جداول حقائق مستقلة تشترك في بُعدي العميل والزمن.

(Architecture Diagram) الرسم المعماري (2)



✓ المكونات التقنية.

- **Source:** SQLite (Kaggle file).
- **ETL:** python and pandas and SQLAlchemy.
- **Target DWH:** PostgreSQL.
- **Reporting:** Direct SQL Queries

3. نمذجة البيانات (Data Modeling)

✓ تحديد العمليات التجارية (Business Processes)

بعد دراسة مخطط المصدر، استخلصنا أربع عمليات تجارية رئيسية، كل منها يقابل جدول حقائق مستقل:

Business Processes	Fact Table	Granularity
sales	fact_sales	بند الطلب (order_id+order_item_id)
order fulfillment	fact_order	One completed order (order_id)
payments	fact_payments	One payment installment (order_id + sequential)
lead conversion	Fact_leads	One qualified business lead (mql_id)

لماذا جداول منفصلة؟

- عملية البيع تتم على مستوى البند (سعر، منتج، بائع)، بينما التقييم وموعد التسليم على مستوى الطلب. دمجها يؤدي إلى تكرار التقييم لكل بند، مما يفسد حسابات المتوسط.
- المدفوعات يمكن أن تكون متعددة للطلب الواحد (أقساط)، فلو دمجت مع المبيعات لأدت إلى مضاعفة الإيرادات خطأً.
- تحويل العملاء المحتملين مجال مختلف يتعلق بالتسويق ولا يرتبط مباشرة بالمبيعات إلا عبر البائع، لذا يُفصل.

4. Dimension Table

جميع جداول الحقائق تتشارك أبعاداً مطابقة (Conformed Dimensions) ، مما يضمن تحليلاً موحداً عبر العمليات.

Dimension	Type	Surrogate Key	Notes
Dim_time	static	Time_key	Pre-populated date dimension with attributes (year, month, weekday).
Dim_customer	SCD2	Customer_key	Tracks changes in customer location over time.
Dim_seller	SCD2	Seller_key	Tracks seller details (city, state) changes.
Dim_product	SCD2	Product_key	Tracks product category and physical attributes changes.
Dim_order_status	Type1	Status_key	Static lookup (delivered, shipped, canceled...).
Dim_payment_type	Type1	Payment_type_key	Static lookup (credit_card, boleto...).
Dim_business_segment	Type1	Segment_key	Static lookup for lead business segment.
Dim_lead_type	Type1	Lead_type_key	Static lookup for lead type.

استراتيجية الأبعاد المتغيرة ببطء: (SCD)

- النوع 2 (SCD Type 2) للأبعاد dim_customer و dim_seller و Dim_product لنتمكن من تتبع تاريخ التغييرات. مثلاً، إذا انتقل عميل إلى مدينة جديدة، فإن مبيعاته السابقة تُنسب إلى المدينة القديمة وقت الطلب.
- النوع 1 (Type 1) للأبعاد الثابتة (حالات، وسائل دفع، قطاعات) لأن قيمها لا تتغير عملياً، ولا حاجة لحفظ تاريخها.

تنفيذ: SCD2:

يحتوي كل بُعد متغير على أعمدة:

- effective_start_date: تاريخ بدء سريان النسخة.
- effective_end_date: تاريخ نهاية سريانها (NULL) للنسخة الحالية.
- is_current: علامة منطقية (TRUE) للنسخة الحالية.

مع فهرس فريد جزئي (Partial Unique Index) لضمان وجود نسخة حالية واحدة فقط لكل مفتاح طبيعي.

```
CREATE UNIQUE INDEX uq_customer_current ON dim_customer(customer_id) WHERE is_current = TRUE;
```

5. Data Pipeline (ETL)

- ❖ **لأسلوب:** تحميل كامل (Full Load) دفعة واحدة. مجموعة البيانات متوسطة الحجم (حوالي 100 ألف طلب) ويمكن تحميلها بالكامل في الذاكرة. يمكن إضافة تحميل ترايدي (Incremental) مستقبلاً باستخدام طابع زمني.
- ❖ **الأدوات:** لغة Python 3 مع مكتبات pandas: للتحويلات، SQLAlchemy و psycopg2 للاتصال بقاعدة PostgreSQL، و logging للتسجيل الأخطاء والتقدم.
- ❖ **بيئة التشغيل:** دفتر Jupyter (IPYNB) لضمان الشفافية وإمكانية إعادة التشغيل خطوة بخطوة.

6. مرحلة خط الانابيب

- الاستخراج (Extract)

❖ الاتصال بقاعدة SQLite باستخدام sqlite3 وقراءة جميع الجداول إلى DataFrames.

○ التحويل(Transform)

- ❖ **تنظيف البيانات:** معالجة القيم المفقودة، تنسيق النصوص، تحويل التواريخ، دمج الإحداثيات والترجمة.
- ❖ **تحميل الأبعاد الثابتة:** استخراج القيم الفريدة وإدراجها في جداول الأبعاد الثابتة مثل Dim_order_status.
- ❖ **تحميل الأبعاد المتغيرة (SCD2):** في التحميل الأولي، تُدرج جميع السجلات كإصدارات حالية مع تاريخ بدء '2016-01-01'.
- ❖ **بناء خرائط المفاتيح البديلة:** استعلام عن الأبعاد الحالية من PostgreSQL وإنشاء قاموس Python يربط المفتاح الطبيعي بالمفتاح البديل.
- ❖ **تجميع جداول الحقائق:**

- ✓ دمج الجداول المصدرية (مثلاً orders مع order_items للمبيعات).
- ✓ استبدال المفاتيح الطبيعية بالمفاتيح البديلة باستخدام القواميس.
- ✓ حساب المقاييس المشتقة (total_item_value, delivery_days, converted).
- ✓ حذف الصفوف التي فشل فيها الربط (نادر جداً).

○ التحميل(load)

- ❖ استخدام DataFrame.to_sql () مع البارامتر if exists='append' لإدراج البيانات بسرعة.
- ❖ تُحمّل الأبعاد أولاً ثم الحقائق لضمان وجود المفاتيح البديلة المشار إليها.

○ معالجة الفشل وقابلية إعادة التشغيل

- ❖ **تسجيل الأحداث (logging):** يسجل البرنامج تقدم كل مرحلة والأخطاء إن حدثت.
- ❖ **إمكانية إعادة التشغيل:** يمكن إفراغ جداول الحقائق وإعادة تشغيل الدفتر من البداية أو من خلية محددة دون تأثير على الأبعاد.
- ❖ **قابلية التوسع:** لو زاد حجم البيانات، يمكن قراءة الملفات على دفعات (chunks) أو استخدام COPY في PostgreSQL بدلاً من to_sql.

7. Performance Optimization

الفهارس (Indexes)

- ❖ مفاتيح أساسية على جميع مفاتيح الأبعاد والحقائق المركبة، مما يوفر فهارس B-tree تلقائية للوصلات السريعة.
- ❖ فهارس ثانوية على المفاتيح الخارجية في جداول الحقائق:
- fact_sales: فهارس على Customer_key، Product_key، Seller_key، order_time_key، بالإضافة إلى فهرس مركب (Product_key, order_time_key) للاستعلامات الشائعة التي تجمع المنتج مع الزمن.
- fact_order: فهارس على Customer_key، order_time_key.
- fact_payments: فهارس على Customer_key، Payment_type_key، Payment_type_key.
- Fact_leads: فهارس على Seller_key.

التقسيم (Partitioning)

- ❖ لم يُنفذ في المخطط الحالي، ولكن يُوصى بتقسيم النطاق حسب السنة (Range Partitioning) على عامود order_time_key في جدول fact_sales و fact_payments لتحسين أداء استعلامات النطاق الزمني.

5.3 المشاهد المادية (Materialized Views)

المشاهد المادية

- ❖ تم تعريف مشهد مادي mv_daily_sales لتجميع المبيعات اليومية مسبقاً، مما يسرع لوحات المعلومات.
- قرارات التصميم الفيزيائي
- استخدمنا قيود REFERENCES (Foreign Keys) بشكل اختياري؛ حيث يضمن الـ ETL التكامل المرجعي، وإلغاء القيود يسرع الإدراج الجماعي.

✓ الخاتمة

تم بنجاح تصميم وتنفيذ مستودع بيانات يفي بمتطلبات التكليف. استخدمنا مخطط المجرة (**Galaxy Schema**) لضمان فصل العمليات التجارية مع الحفاظ على التكامل عبر الأبعاد المطابقة. خط الأنابيب يتسم بالمتانة وقابلية إعادة التشغيل، مع تحسينات أداء مناسبة. القرارات الهندسية موثقة ومبررة، مما يجعل الحل جاهزاً للإنتاج وقابلاً للتوسع المستقبلي.